

Implementación de Impresión y Validación de Tickets

Actividad N°: 24 **Fecha:** 02/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar el flujo de impresión de boletos y su reimpresión controlada, con las validaciones de datos correspondientes (HU-05 y RF-07).

2. Impresión (**printTicket**)

```
POST /api/tickets/:id/print { quienRetira, quienOtro?, parentesco?, celular }
|
▼
Validar: quienRetira y celular obligatorios
|
▼
Si quienRetira = "Otro": validar quienOtro y parentesco obligatorios
|
▼
Buscar ticket por 'Ticket ID' → 404 si no existe
|
▼
{ticket.impreso === true Y usuario NO es Jefe? —Sí→ 400 "Este ticket ya fue impreso"
| No (o es Jefe reimprimiendo)
▼
Actualizar: impreso=true, fechaImpresion=now, usuarioResponsable, puntoTrabajo,
quienRetira, celular (+ quienOtro/parentesco si aplica)
|
▼
ticket.save()
|
▼
Emitir Socket.IO 'ticket-updated' (action: 'print')
|
▼
200 { ticket actualizado }
```

Nota de diseño: a diferencia del canje (**canjeTicket**, que bloquea totalmente un segundo intento), la impresión permite que un **Jefe** reimprima directamente desde el mismo endpoint si es necesario, mientras que un Staff solo puede imprimir una vez. Esto se complementa con el endpoint dedicado de reimpresión para dejar registro explícito del motivo.

3. Reimpresión controlada (**reprintTicket**)

```

POST /api/tickets/:id/reprint { motivo }
    |
    ▼
Validar motivo obligatorio
    |
    ▼
Buscar ticket → 404 si no existe
    |
    ▼
{ticket.impreso === false? —Sí→ 400 "No se puede reimprimir un ticket que no
ha sido impreso"
    | No (ya fue impreso antes)
    ▼
Agregar entrada al arreglo ticket.reimpresiones:
  { fecha: now, motivo, usuario: req.user._id, puntoTrabajo: req.user.puntoTrabajo
}
    |
    ▼
ticket.save()
    |
    ▼
200 { ticket con historial de reimpresiones actualizado }

```

Acceso restringido exclusivamente a `authorize('jefe')` en `routes/tickets.js`, consistente con la regla de negocio validada desde la Semana 1 (RF-07).

4. Validación de datos de retiro (regla común a impresión y canje)

Campo	Regla
<code>quienRetira</code>	Obligatorio; debe ser <code>Titular</code> , <code>Titular Compra</code> u <code>Otro</code>
<code>celular</code>	Obligatorio siempre
<code>quienOtro</code>	Obligatorio solo si <code>quienRetira = "Otro"</code>
<code>parentesco</code>	Obligatorio solo si <code>quienRetira = "Otro"</code> ; valores permitidos: Esposo/a, Hijo/a, Padre/Madre, Hermano/a, Amigo/a, Otro parentesco

Esta validación existe tanto en el frontend (formulario, para feedback inmediato) como en el backend (controlador y esquema Mongoose con `validate` condicional), de modo que no se puede omitir enviando la petición directamente a la API.

5. Historial embebido de reimpresiones

Se optó por modelar `reimpresiones` como un arreglo de subdocumentos dentro del propio `Ticket` (no como una colección aparte), porque:

- Su ciclo de vida depende completamente del ticket padre (no tiene sentido una reimpresión "huérfana").
- Se consulta casi siempre junto con el ticket (al ver el detalle de un boleto), no de forma independiente.

- El volumen esperado de reimpresiones por ticket es bajo (excepcional, no la operación principal).

6. Conclusiones del día

Quedan implementados los flujos de impresión y reimpresión controlada, reutilizando las mismas reglas de validación de datos de retiro definidas para el canje, y manteniendo el historial de reimpresiones trazable dentro de cada boleto.

Observaciones: Sin observaciones.